

INGENIERÍA DE SOFTWARE II
CASO
Sócrates Rojas Amador
VERSIÓN 1.0



Caso de estudio: Sistema integrado de triaje y disponibilidad de camas (sit-cama)

Institución: Clínica San Rafael de Occidente (IPS Privada de Nivel II de Complejidad, Colombia)
Modalidad del proyecto: Aplicación Híbrida (Web PWA / Capacitor para Android/iOS)
Ventana de ejecución: Timebox estricto de 2 semanas (10 días hábiles)
Marco de trabajo: Modelo Híbrido RUP/UP (Gobernanza de Fases e Hitos) + Scrum (Ejecución táctica de iteraciones) + DevSecOps / ALM con GitLab/GitHub

1. Contexto operativo y problema de negocio

La Clínica San Rafael de Occidente atiende urgencias de mediana complejidad (pediatría, medicina interna, cirugía general, gineco-obstetricia y trauma básico). Actualmente, el registro de triaje de urgencias y la asignación de camas se realiza mediante planillas físicas o un aplicativo monolítico de escritorio no accesible en salas o ambulancias.

Desafíos clave

1. Cuellos de botella en urgencias: Retrasos en la categorización de pacientes según la Resolución 5596 de 2015 del Ministerio de Salud de Colombia (Triaje I al V).

2. Falta de visibilidad en tiempo real: Las camas hospitalarias desocupadas tardan hasta 3 horas en reflejarse en el sistema asistencial.

3. Cumplimiento normativo: Protección estricta de datos sensibles (Habeas Data - Ley Estatutaria 1581 de 2012) y trazabilidad clínica conforme a la Resolución 3100 de 2019 (Habilitación de servicios de salud).

2. Definición del MVP y alcance (Timebox: 2 semanas)

Para entregar un producto funcional en 10 días hábiles, el alcance se acota a dos módulos asistenciales críticos:

Módulo A: Triaje digital rápido
- Clasificación I a V (Res. 5596)

Módulo B: Censo dinámico de camas
- Visualización de camas por pabellón

- Signos vitales y motivo de consulta
- Generación de alerta médica inmediata
- Estados: Disponible, Ocupada, Desinfección
- Asignación directa desde triaje (I y II)

Arquitectura Técnica

- Frontend híbrido: Ionic + Angular o React con Capacitor (Web responsive para estaciones de enfermería y APK Android/iOS para tablets y médicos en ronda).
- Backend: Node.js (NestJS) o Go (Fiber) con arquitectura limpia / hexagonal en contenedores Docker.
- Persistencia: PostgreSQL con esquemas aislados y cifrado en reposo para datos sensibles del paciente.

3. Gobernanza Metodológica: RUP Estándar + Scrum

Se utiliza la gobernanza de fases e hitos de UP/RUP como marco estructural corporativo, y Scrum con cadencia diaria para la construcción del software.

Fase RUP	Duración	Hito RUP	Ceremonias Scrum y Entregables
Iniciación (Inception)	Días 1 y 2	LCO (Lifecycle Objective)	Sprint 0: Definición de Épicas, Product Backlog en ALM, Arquitectura de referencia, Criterios de Aceptación y DoD.
Elaboración (Elaboration)	Días 3 y 4	LCA (Lifecycle Architecture)	Sprint 1 (Inicio): Despliegue de la Línea Base Arquitectónica Ejecutable, Pipeline CI/CD conectado y autenticación RBAC operativa.
Construcción (Construction)	Días 5 al 8	IOC (Initial Operational Capability)	Sprint 1 (Cierre) & Daily Standups: Desarrollo modular de Triaje y Camas, pruebas unitarias automáticas y análisis SAST/SCA.
Transición (Transition)	Días 9 y 10	PR (Product Release)	Sprint Review & Retrospective: Pruebas de carga, DAST, validación UAT médica, firma de contenedores y despliegue a Staging/Producción.



INGENIERÍA DE SOFTWARE II
CASO
Sócrates Rojas Amador
VERSIÓN 1.0



Requisitos Orientados a UP (Unified Process)

En UP, las necesidades del cliente se capturan a través del modelo FURPS+ (Funcionalidad, Usabilidad, Confiabilidad, Rendimiento y Soporte/Restricciones) y se formalizan mediante Especificaciones de Casos de Uso y Requisitos Suplementarios.

1. **Requisitos Funcionales del Cliente (RF - Base para Casos de Uso)**
    - RF-01 (Admisión y Triage Normativo): El sistema debe permitir a enfermería ingresar los datos de identificación del paciente y sus signos vitales: Presión Arterial (sistólica/diastólica), Frecuencia Cardíaca, Frecuencia Respiratoria, Saturación de Oxígeno (SpO_2), Escala de Coma de Glasgow y Nivel de Dolor (1 al 10).

    - RF-02 (Cálculo Automático de Triage): El software debe aplicar el algoritmo de clasificación según la Resolución 5596 de 2015 del Ministerio de Salud de Colombia:

        - Triage I (Resucitación): SpO_2 < 85%, Glasgow < 9, o Paro Cardiorrespiratorio inminente (atención inmediata, 0 min).

        - Triage II (Emergencia): Dolor precordial típico, Glasgow 9-13, dolor severo > =8 (tiempo de atención <= 30 min).

        - Triage III a V (Urgencia menor a no urgente): Tiempos de atención diferibles (hasta 120-240 min).

    - RF-03 (Alerta Inmediata de reanimación): Si el triage calculado es nivel I o II, el sistema debe disparar un evento de notificación acústica y visual en el panel general de la estación de médicos.

    - RF-04 (Censo dinámico de camas): El personal de admisiones y hospitalización debe visualizar el estado de las camas agrupadas por servicio: Urgencias Adultos (15 camas), Urgencias Pediátricas (8 camas), Observación (12 camas) y Aislamiento (3 camas).
    - RF-05 (Ciclo de estados de cama): Cada cama debe soportar la máquina de estados:
        Disponible
        Ocupada
        En Limpieza/Desinfección
        Disponible.

- RF-06 (Asignación directa de paciente): Permitir vincular un paciente en Triage I o II a una cama en estado Disponible con un único clic o toque de pantalla, cambiando automáticamente la cama a Ocupada.

- RF-07 (Control de acceso basado en roles - RBAC): Restringir acciones por perfil: Enfermero triador (crea triage), Médico de urgencias (atiende y ordena cama), Camillero/Servicios generales (cambia estado a Limpieza/Disponible), Auditor médico (solo lectura de reportes).

2. Requisitos No Funcionales del Cliente (FURPS+ / Suplementarios)

Usabilidad (Usability):

- El formulario de triage móvil debe diligenciarse en menos de 90 segundos en tablets asistenciales con guantes quirúrgicos (botones táctiles mínimos de 48 x 48 px).

Rendimiento (Performance):

- Tiempo de respuesta de la API para el cálculo de triage menor a 300 milisegundos.
- Propagación del cambio de estado de una cama a todos los clientes conectados en menos de 1 segundo.
-

Confiabilidad (Reliability):

- Disponibilidad del servicio 24/7 de al menos el 99.5%. En caso de pérdida de red WiFi en sala, la aplicación móvil debe conservar el borrador en almacenamiento local seguro (IndexedDB/SQLite) y sincronizar al reconectar.

Soporte y restricciones (Supportability & Constraints):

- Marco Legal Colombiano: Cumplimiento de la Ley Estatutaria 1581 de 2012 (Habeas Data) y Resolución 3100 de 2019 (Habilitación de servicios de salud). Los datos sensibles del paciente (nombre, cédula, diagnóstico) deben estar cifrados en reposo con AES-256.
- Auditoría: Registro de logs inmutables (timestamp, usuario, IP, acción previa y posterior) para toda modificación de triage o cama.

Guía de artefactos UP que el estudiante debe construir

Con estos requisitos, el estudiante debe generar en el ALM (GitLab/GitHub):

1. Documento de visión: Planteamiento del problema, stakeholders, objetivos del sistema y límites de la solución.

2. Modelo de casos de uso: Diagrama general de Casos de Uso en UML delimitando actores (Enfermero, Médico, Servicios Generales).
3. Especificaciones de casos de uso expandidos: Detalle paso a paso de CU-01: Clasificar Triage y CU-02: Gestionar Estado de Cama (Flujo básico, flujos alternativos, condiciones de error, pre y post-condiciones)
4. Especificación suplementaria: Catálogo FURPS+ mapeado a los requisitos legales colombianos.
5. Documento de arquitectura de software (SAD): Vistas 4+1 (Diagrama de paquetes, diagrama de secuencia de triage y diagrama de despliegue en contenedores).